



الجمهورية العربية السورية
وزارة التعليم العالي جامعة دمشق
كلية الهندسة المعلوماتية

محرك المعالجة عالي الاداء لنظام التجارة الالكترونية
(High-Performance E-commerce Backend
Engine)

مقرر: البرمجة التفرعية

اعداد الطلاب:

- مادح هاشم معاد
- مجد محمد محمد
- عمر فادي العيسى

مقدمة :

يهدف هذا المشروع إلى بناء نظام تجارة إلكترونية باستخدام إطار العمل Laravel مع تطبيق مجموعة من مفاهيم البرمجة المتوازية والأنظمة الموزعة لمعالجة التحديات المرتبطة بالتزامن والأداء وقابلية التوسع. وقد تم التركيز على تنفيذ حلول عملية لمشكلات الوصول المتزامن إلى الموارد المشتركة، ومنع حدوث حالات التضارب (Race Conditions) ، وضمان سلامة المعاملات (Transaction Integrity)، بالإضافة إلى تحسين الأداء من خلال استخدام تقنيات التخزين المؤقت (Caching) وتوزيع الأحمال (Load Balancing).

كما يتضمن المشروع تطبيق آليات للتحكم بالتزامن باستخدام الأقفال الموزعة، ومحاكاة بيئة متعددة المستخدمين لاختبار سلوك النظام تحت الضغط، مع قياس الأداء وتحليل الاختناقات المحتملة واقتراح الحلول المناسبة لمعالجتها. وقد تم استخدام مجموعة من الأدوات والتقنيات الحديثة مثل Laravel ، MySQL ، Redis ، و Apache JMeter لتحقيق هذه الأهداف والتحقق من فعالية الحلول المطبقة.

يعرض هذا التقرير تفاصيل تصميم النظام وآليات التنفيذ والاختبارات التي تم إجراؤها، بالإضافة إلى النتائج التي تم الحصول عليها لإثبات قدرة النظام على التعامل مع العمليات المتزامنة بكفاءة مع المحافظة على صحة البيانات واستقرار الأداء.

المتطلب السادس

استراتيجية التخزين المؤقت (Caching Distributed)

وصف المشكلة

تعتمد بعض أجزاء النظام على عمليات قراءة متكررة للبيانات نفسها، مثل قائمة المنتجات والمتاجر والمفضلة الخاصة بالمستخدمين. يؤدي تنفيذ الاستعلامات نفسها بشكل متكرر إلى زيادة الحمل على قاعدة البيانات وارتفاع زمن الاستجابة، خصوصًا عند ازدياد عدد الطلبات الواردة للنظام.

لذلك كان من الضروري إضافة طبقة تخزين مؤقت تسمح بإعادة استخدام البيانات المطلوبة بشكل متكرر بدلاً من إعادة تنفيذ الاستعلامات في كل مرة.

الاستراتيجية المستخدمة

تم اعتماد Redis كطبقة تخزين مؤقت (Caching Layer) داخل النظام، حيث يتم الاحتفاظ بنتائج الاستعلامات الأكثر استخدامًا لفترة زمنية محددة وإعادتها مباشرة عند الطلبات اللاحقة.

تم تطبيق Redis على أكثر الموارد استخدامًا داخل التطبيق، وهي:

- عرض المنتجات .
- عرض تفاصيل المنتج .
- عرض قائمة المفضلة الخاصة بالمستخدم .

كما تم اعتماد سياسة إبطال الكاش (Cache Invalidation) عند إضافة أو تعديل أو حذف البيانات المرتبطة بهذه الموارد لضمان بقاء البيانات المعروضة للمستخدم محدثة وصحيحة.

آلية التنفيذ

عند طلب البيانات يقوم النظام أولاً بالتحقق من وجود نسخة محفوظة داخل Redis.

في حال وجود البيانات يتم إرجاعها مباشرة من الكاش دون الوصول إلى قاعدة البيانات.

أما إذا لم تكن البيانات موجودة داخل Redis فيتم تنفيذ الاستعلام على قاعدة البيانات ثم تخزين النتيجة داخل الكاش وإعادتها للمستخدم.

وعند حدوث أي تعديل على البيانات المرتبطة بهذه الموارد يتم حذف الكاش القديم ليتم إنشاء نسخة جديدة محدثة عند الطلب التالي.

التقنيات المستخدمة

- Laravel Cache System
- Redis
- Cache::remember()
- Cache::forget()
- Apache JMeter

الاختبار

تم التحقق من عمل Redis Cache من خلال:

1. تنفيذ طلب على المورد لأول مرة ومراقبة إنشاء المفتاح داخل Redis.
2. إعادة تنفيذ الطلب نفسه للتأكد من استرجاع البيانات من الكاش .
3. تنفيذ عمليات إنشاء وتعديل وحذف للتحقق من حذف الكاش القديم وإعادة بنائه عند الحاجة.

النتائج

نجح النظام في تخزين البيانات الأكثر طلبًا داخل Redis وإعادة استخدامها عند الطلبات اللاحقة، مما قلل عدد الاستعلامات المباشرة على قاعدة البيانات ووفر أساسًا لتحسين الأداء العام للنظام.

```
class ProductController extends Controller
{
    public function index()
    {
        $useCache = true ;

        if ($useCache) {
            $products = Cache::remember('products.all', 60, function () {
                return Product::with('store')->latest()->get();
            });
        } else {
            $products = Product::with('store')->latest()->get();
        }

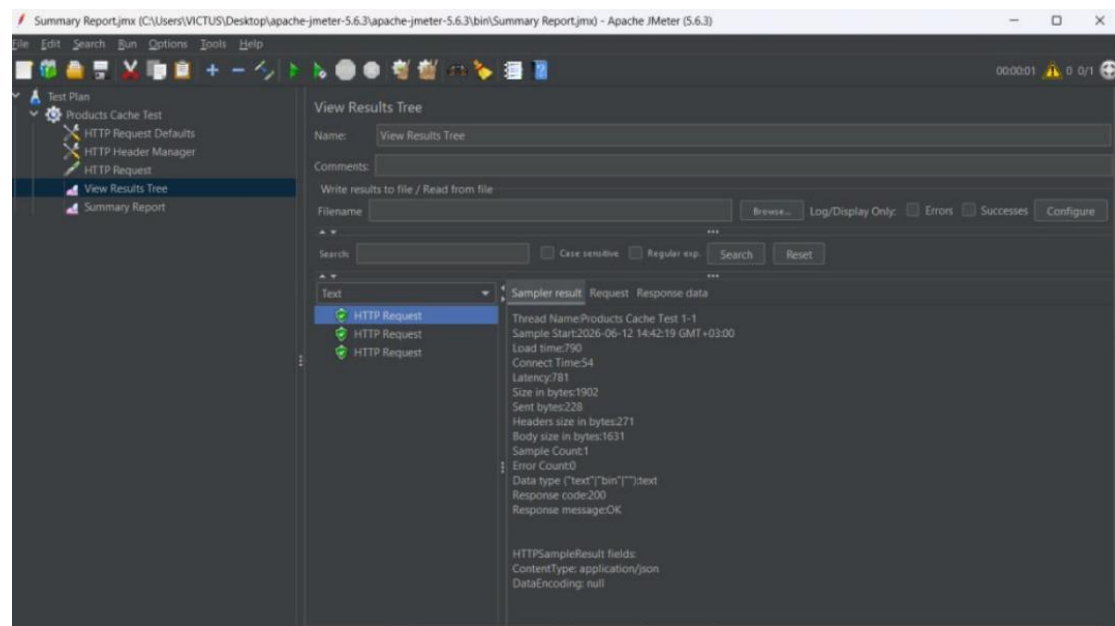
        return response()->json($products);
    }

    public function show(Product $product)
    {
        $cachedProduct = Cache::remember("products.{ $product->id }", 60, function () use ($product) {
            return $product->load('store');
        });

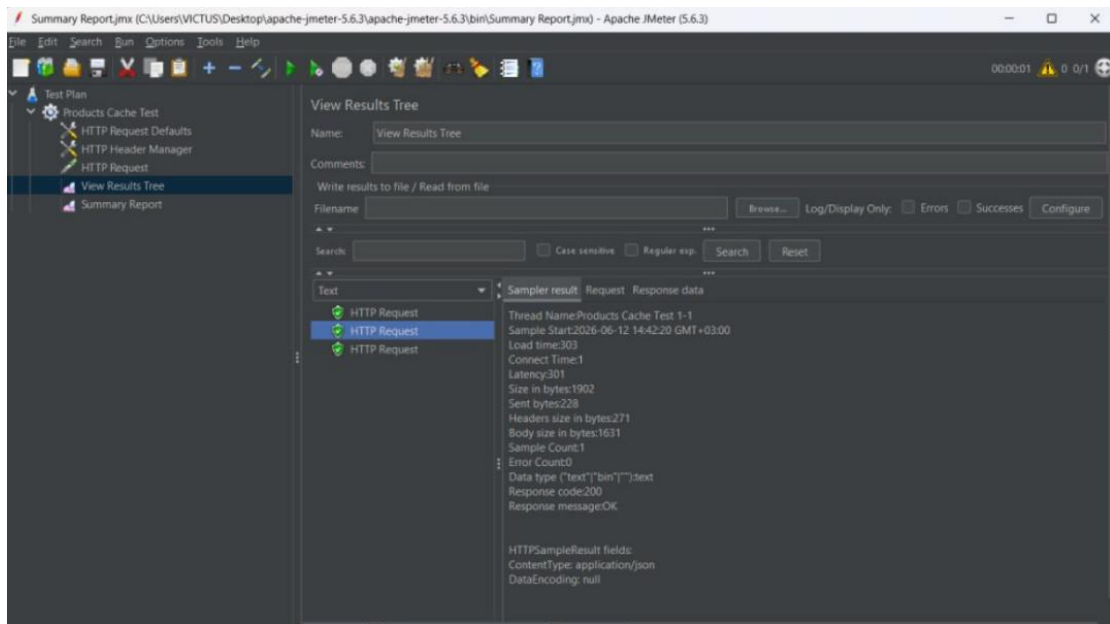
        return response()->json($cachedProduct);
    }
}
```

Cache location in code

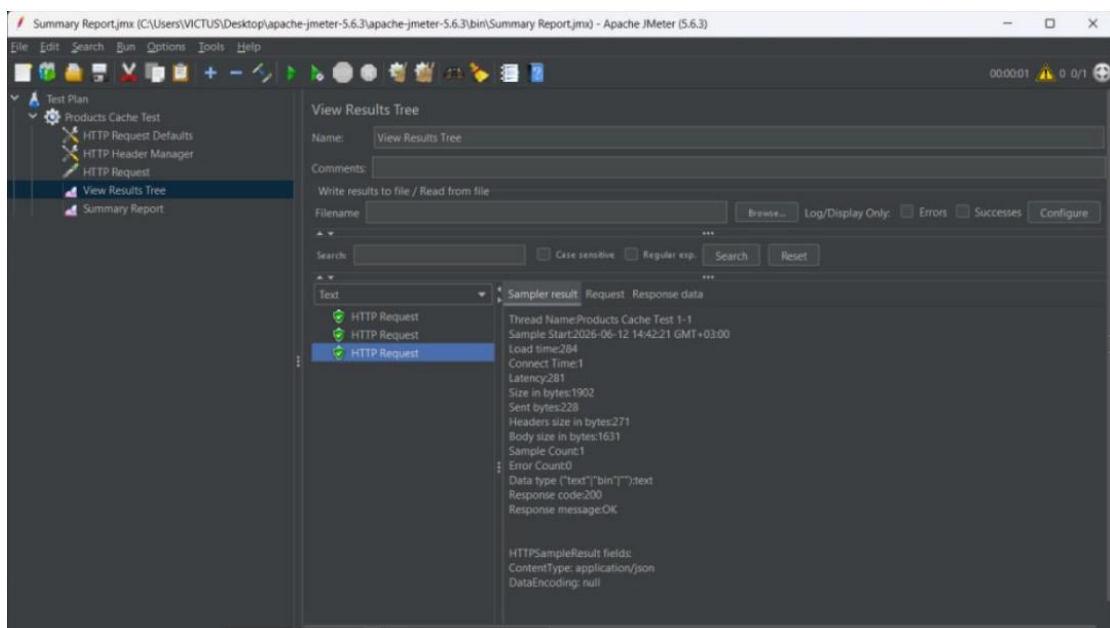
Product cache test photos :



First Request



Second Request



Third Request

نلاحظ انخفاض قيمة ال latency بعد اول طلب لأنه اصبح يأتي بالداتا من ال cache

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
HTTP Reque...	3	459	284	790	234.18	0.00%	2.2/sec	4.04	0.48	1902.0
TOTAL	3	459	284	790	234.18	0.00%	2.2/sec	4.04	0.48	1902.0

☐ Include group name in label? ☒ Save Table Header

Summery Report

Favorite cache test photos :

View Results Tree

Name: View Results Tree

Comments:

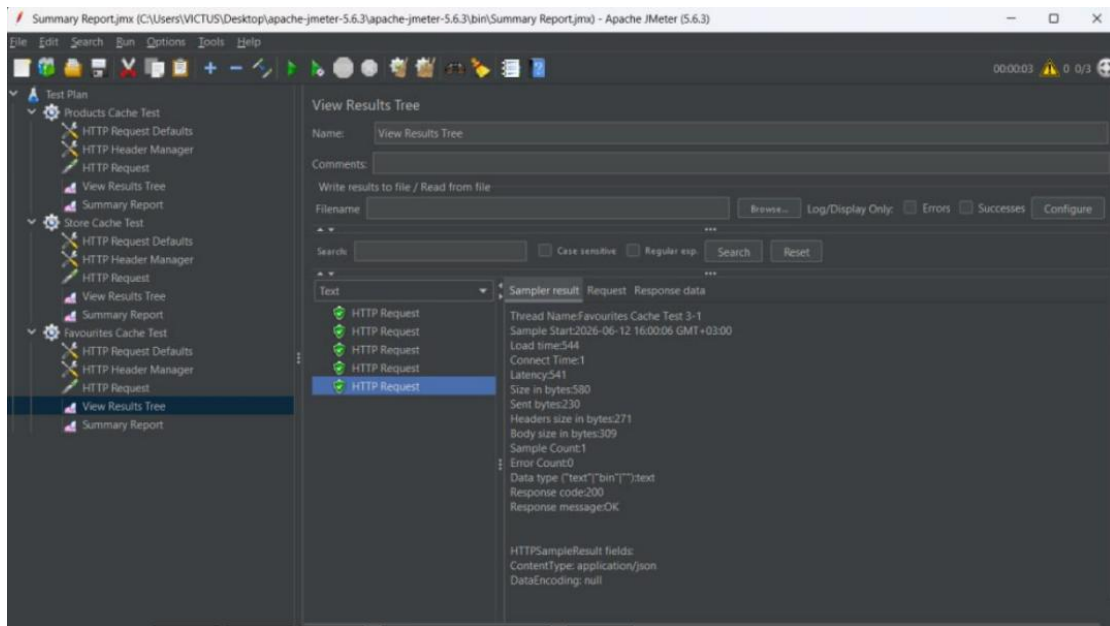
Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

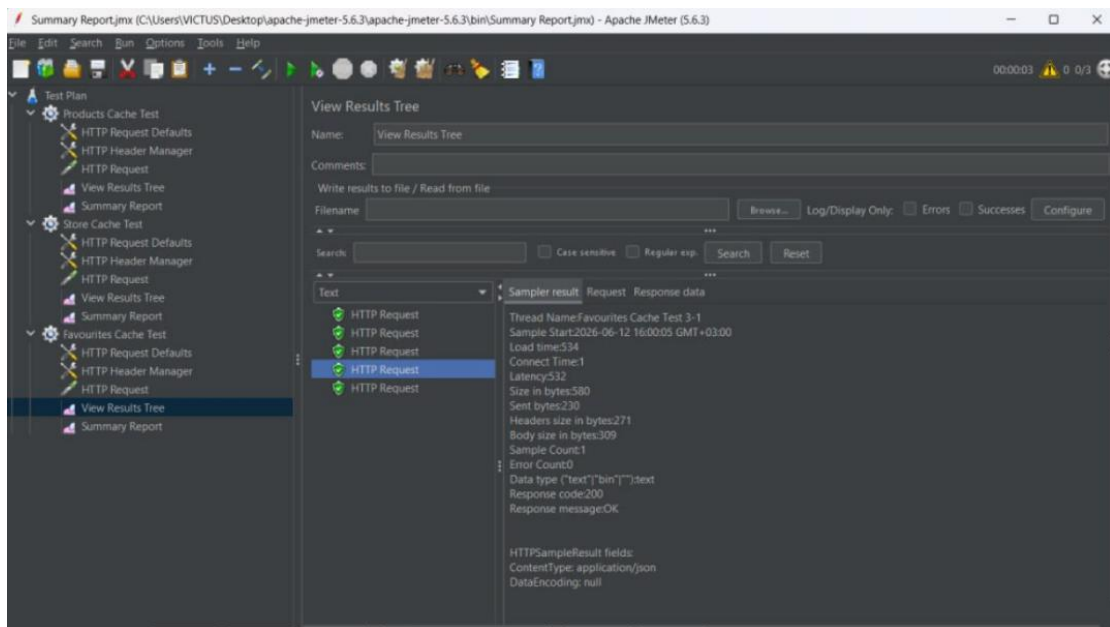
Search: ☐ Case sensitive ☐ Regular exp.

Text	Sampler result	Request	Response data
HTTP Request	Thread Name:Favourites Cache Test 3-1	Sample Start:2026-06-12 16:00:04 GMT+03:00	
HTTP Request	Load time:826		
HTTP Request	Connect Time:2		
HTTP Request	Latency:824		
HTTP Request	Size in bytes:530		
HTTP Request	Sent bytes:230		
HTTP Request	Headers size in bytes:271		
HTTP Request	Body size in bytes:309		
HTTP Request	Sample Count:1		
HTTP Request	Error Count:0		
HTTP Request	Data type ("text","bin"):"text"		
HTTP Request	Response code:200		
HTTP Request	Response message:OK		
HTTP Request	HTTPSampleResult fields:		
HTTP Request	ContentType: application/json		
HTTP Request	DataEncoding: null		

First Request



Second Request



Third Request

نلاحظ انخفاض قيمة ال latency بعد اول طلب لأنه اصبح يأتي بالداتا من ال cache

The screenshot shows the Apache JMeter Summary Report window. The left sidebar lists the test plan structure, including 'Products Cache Test', 'Score Cache Test', and 'Favourites Cache Test'. The main area displays the 'Summary Report' for the selected test. The report includes a table with the following data:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
HTTP Reque...	5	737	534	974	171.73	0.00%	1.4/sec	0.77	0.30	580.0
TOTAL	5	737	534	974	171.73	0.00%	1.4/sec	0.77	0.30	580.0

Summary Report

المتطلب السابع التحكم في الأقفال (Control Concurrency)

وصف المشكلة

تحديث كميات المخزون يمثل مورداً حساساً داخل النظام، حيث يمكن أن يحاول عدة مستخدمين شراء المنتج نفسه في اللحظة ذاتها. في حال عدم وجود آلية مناسبة للتحكم بالتزامن قد يؤدي ذلك إلى تضارب في التحديثات أو السماح ببيع كمية أكبر من المتوفرة فعلياً.

الاستراتيجية المستخدمة

تم استخدام آلية Distributed Lock عبر Redis من خلال Laravel Cache Locks لحجز المورد الحساس أثناء تنفيذ العملية. تسمح هذه الآلية بمرور عملية واحدة فقط على المنتج في الوقت نفسه، بينما تنتظر العمليات الأخرى حتى انتهاء العملية الحالية.

آلية التنفيذ

عند تنفيذ عملية شراء مباشرة أو تأكيد سلة المشتريات يقوم النظام بإنشاء مفتاح قفل مرتبط بالمنتج المطلوب. إذا تمكن الطلب الحالي من الحصول على القفل يتم تنفيذ العملية بالكامل، أما الطلبات الأخرى فتبقى منتظرة حتى تحرير القفل أو انتهاء مهلة الانتظار.

التقنيات المستخدمة

Redis Distributed Lock •

Cache::lock() •

Laravel Framework •

Apache JMeter •

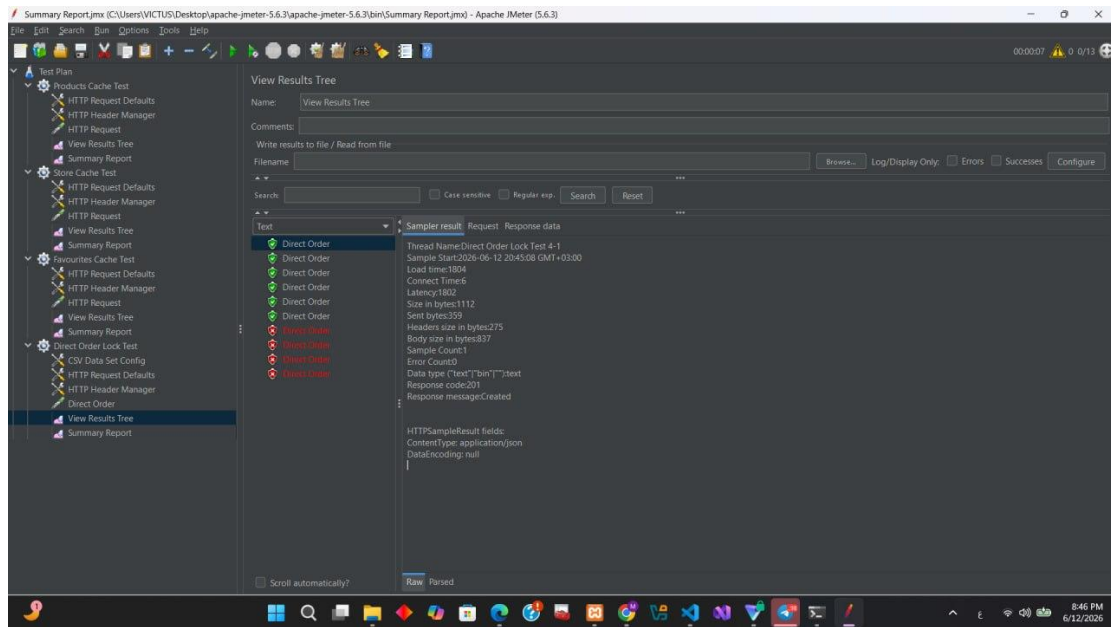
الاختبار

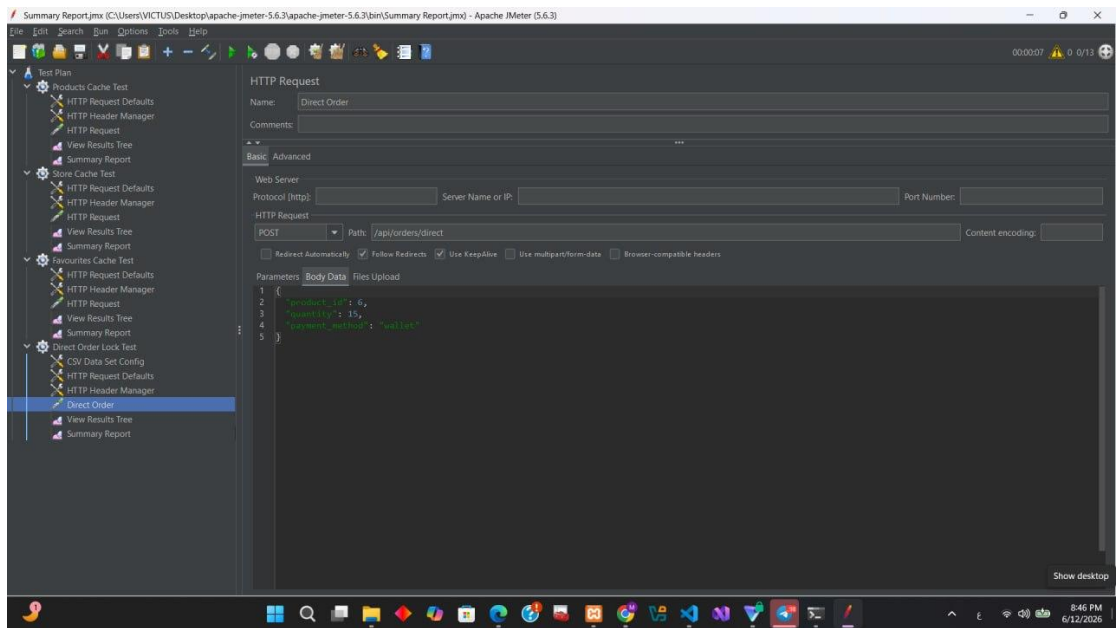
تم إنشاء عدة طلبات متزامنة تستهدف المنتج نفسه للتحقق من أن عمليات التحديث تتم بشكل متسلسل دون تضارب.

النتائج

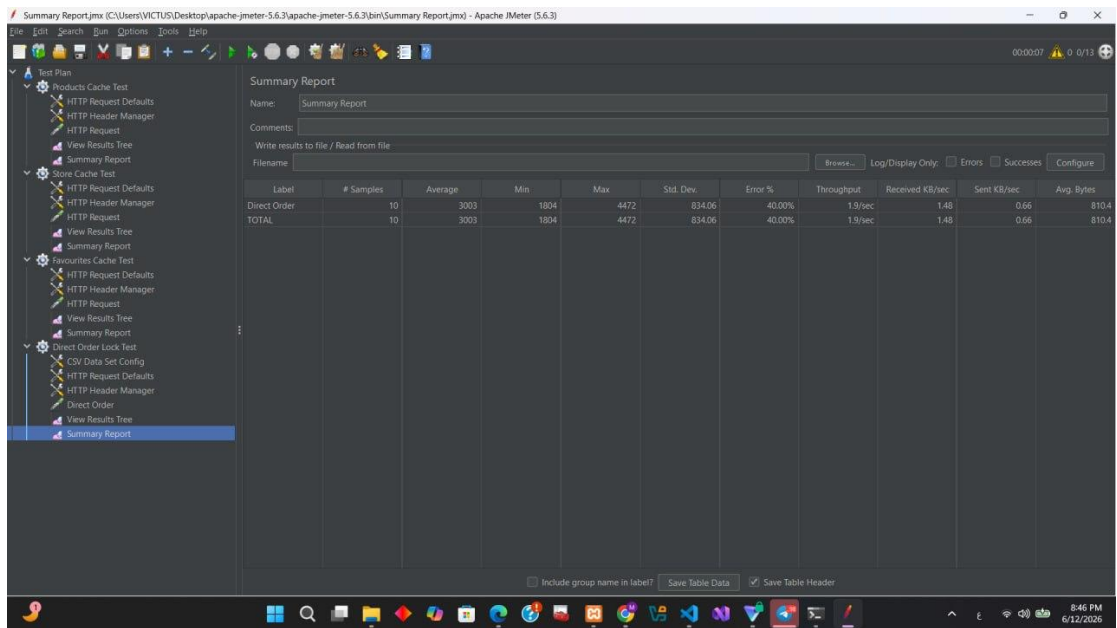
أثبتت النتائج أن النظام يمنع حدوث تضارب أثناء تحديث المخزون، وأن جميع العمليات تمت بصورة صحيحة دون تجاوز الكميات المتاحة أو فقدان الاتساق بين الطلبات المتزامنة.

Direct Order lock test photos :



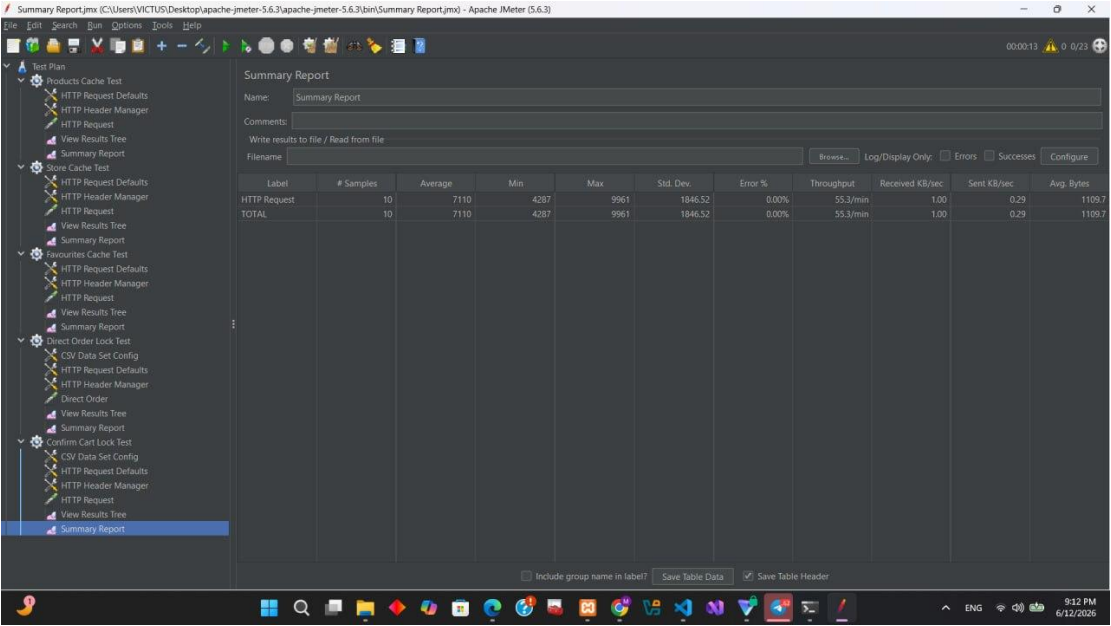
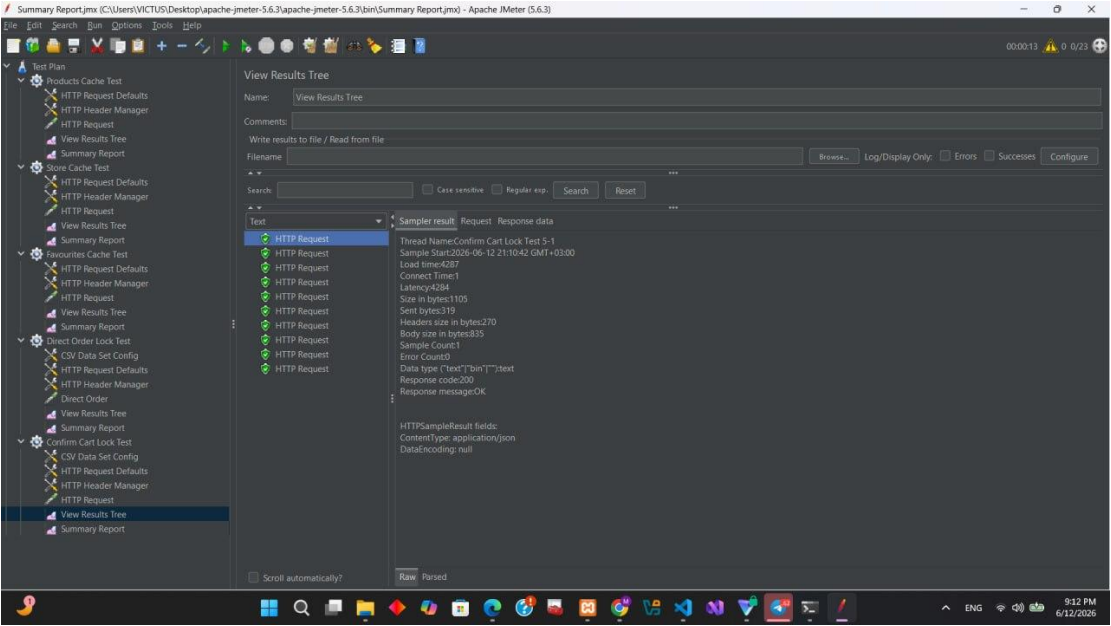


Body Data



Summary Report

Confirm Cart lock test photos :



Summary Report

المتطلب الثامن

سلامة المعاملات (ACID Transaction Integrity)

وصف المشكلة

تتكون عملية الشراء من عدة خطوات مترابطة تشمل التحقق من الرصيد، تحديث كمية المنتج، إنشاء الطلب، وإنشاء سجل الدفع. في حال فشل أي خطوة من هذه الخطوات يجب أن يتم إلغاء جميع التغييرات السابقة للحفاظ على سلامة البيانات.

الاستراتيجية المستخدمة

تم استخدام Database Transactions لضمان تنفيذ العملية المركبة كوحدة واحدة غير قابلة للتجزئة. وفي حال حدوث أي خطأ يتم تنفيذ Rollback وإلغاء جميع التغييرات المنفذة داخل المعاملة.

آلية التنفيذ

تتضمن المعاملة:

1. التحقق من توفر الكمية.
2. التحقق من الرصيد.
3. إنشاء الطلب.
4. إنشاء عناصر الطلب.
5. تحديث المخزون.
6. تحديث رصيد المستخدم.
7. إنشاء سجل الدفع.

إذا فشلت أي خطوة يتم التراجع عن جميع الخطوات السابقة تلقائياً.

التقنيات المستخدمة

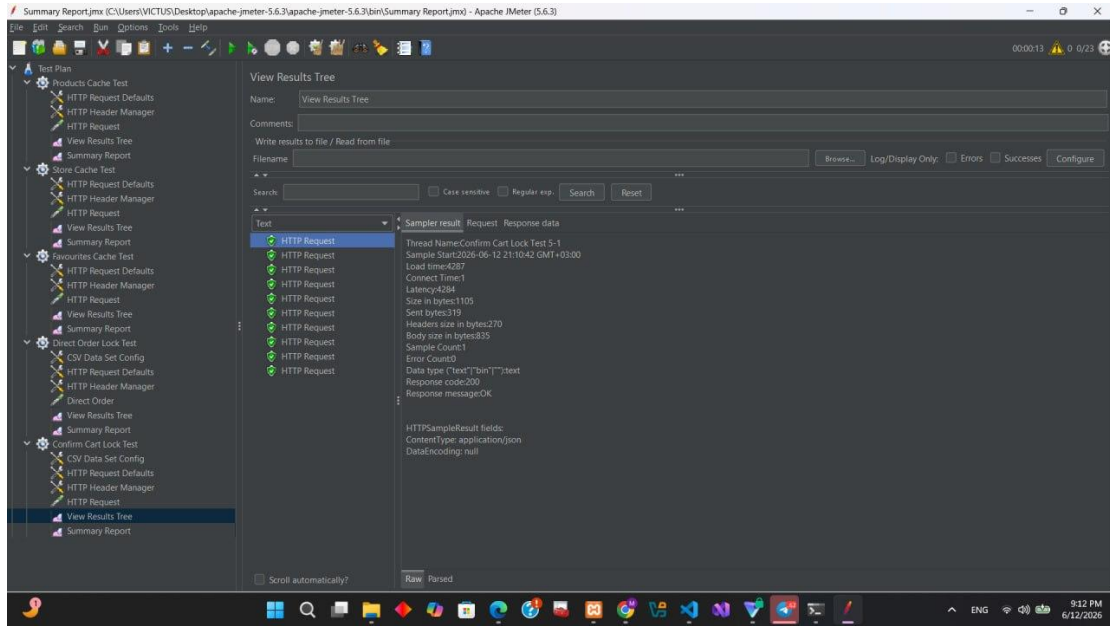
- DB Transactions
- MySQL
- Laravel Framework
- Apache JMeter

الاختبار

تم تنفيذ عدد كبير من الطلبات المتزامنة على عمليات الشراء ومراقبة حالة قاعدة البيانات بعد انتهاء التنفيذ للتأكد من عدم وجود بيانات جزئية أو عمليات غير مكتملة.

النتائج

- أظهرت النتائج أن جميع العمليات إما اكتملت بالكامل أو ألغيت بالكامل دون ترك أي أثر جزئي داخل قاعدة البيانات، مما يحقق مبادئ ACID المتعلقة بالـ Atomicity و Consistency.



المتطلب التاسع

اختبار الاستقرار تحت الضغط (Stress Testing)

وصف المشكلة

يجب التأكد من قدرة النظام على خدمة عدد كبير من المستخدمين المتزامنين دون انهيار أو فقدان للبيانات أو انخفاض حاد في الأداء.

الاستراتيجية المستخدمة

تم تنفيذ مجموعة من اختبارات الضغط باستخدام Apache JMeter لمحاكاة 100 مستخدم متزامن على أهم العمليات الأساسية داخل النظام.

العمليات المختبرة

- GET /api/products
- GET /api/favourites
- POST /api/cart/add
- POST /api/cart/confirm
- POST /api/orders/direct

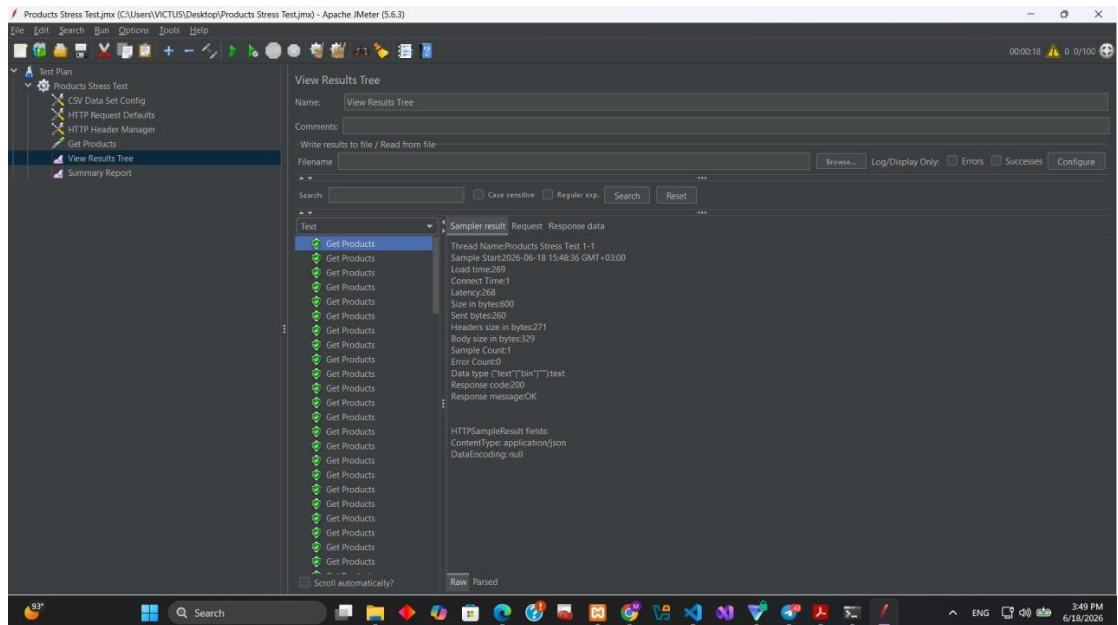
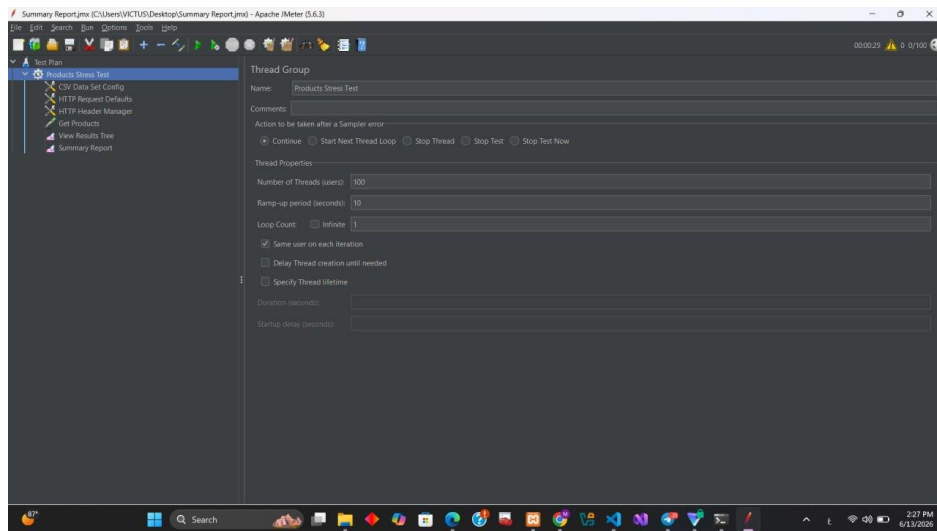
الاختبار

تم إنشاء 100 مستخدم واستخدام Access Tokens مستقلة لهم، ثم تشغيل كل عملية ضمن Thread Group مستقل لمحاكاة الضغط الحقيقي على النظام.

النتائج

أثبتت الاختبارات قدرة النظام على التعامل مع 100 مستخدم متزامن مع المحافظة على صحة البيانات وعدم حدوث انهيار في التطبيق أو فقدان للطلبات.

Products Stress Test :



Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get Products	100	10648	759	19636	5296.45	0.00%	3.4/sec	2.00	0.87	600.0
TOTAL	100	10648	759	19636	5296.45	0.00%	3.4/sec	2.00	0.87	600.0

☐ Include group name in label? ☒ Save Table Header

Summary Report

Favorites Stress Test :

Thread Group

Name: Favourites Stress Test

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users): 100

Ramp-up period (seconds): 10

Loop Count: ☐ Infinite ☒ 1

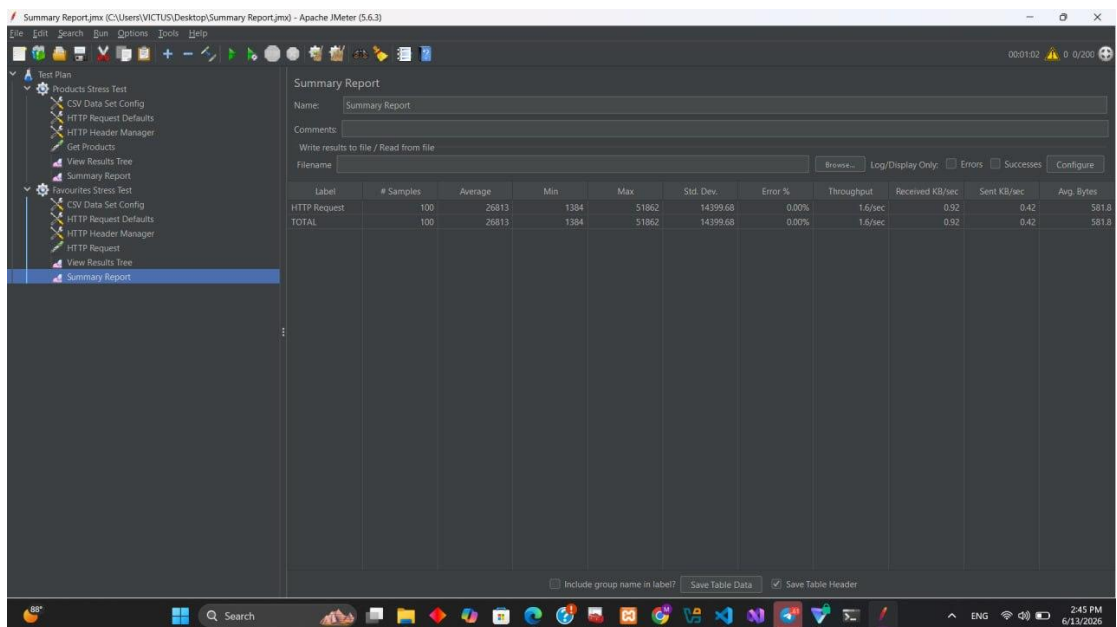
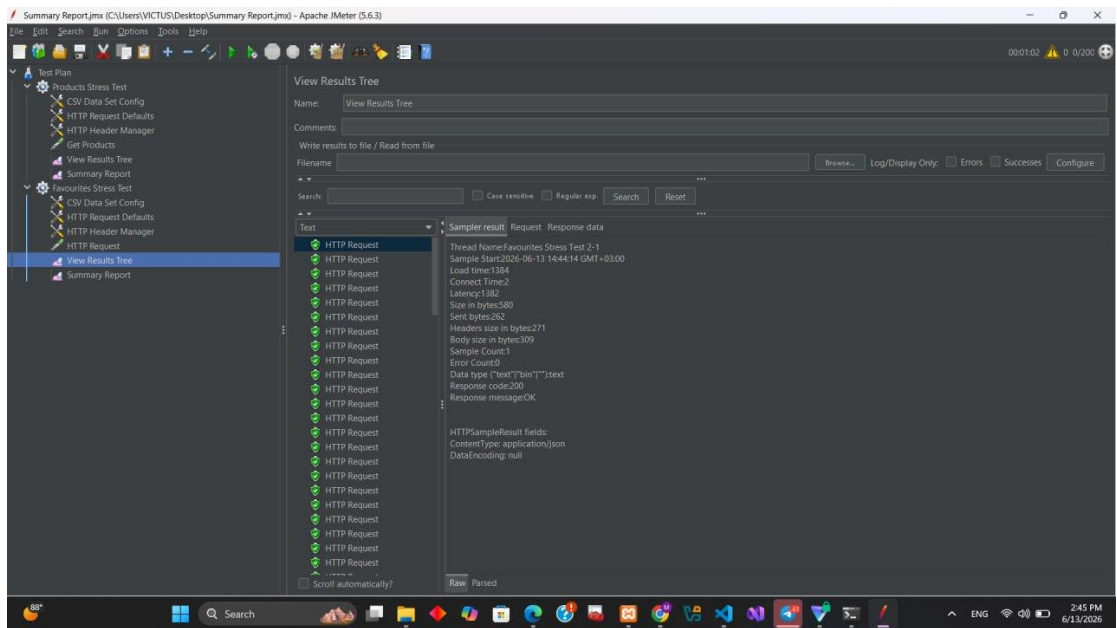
☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

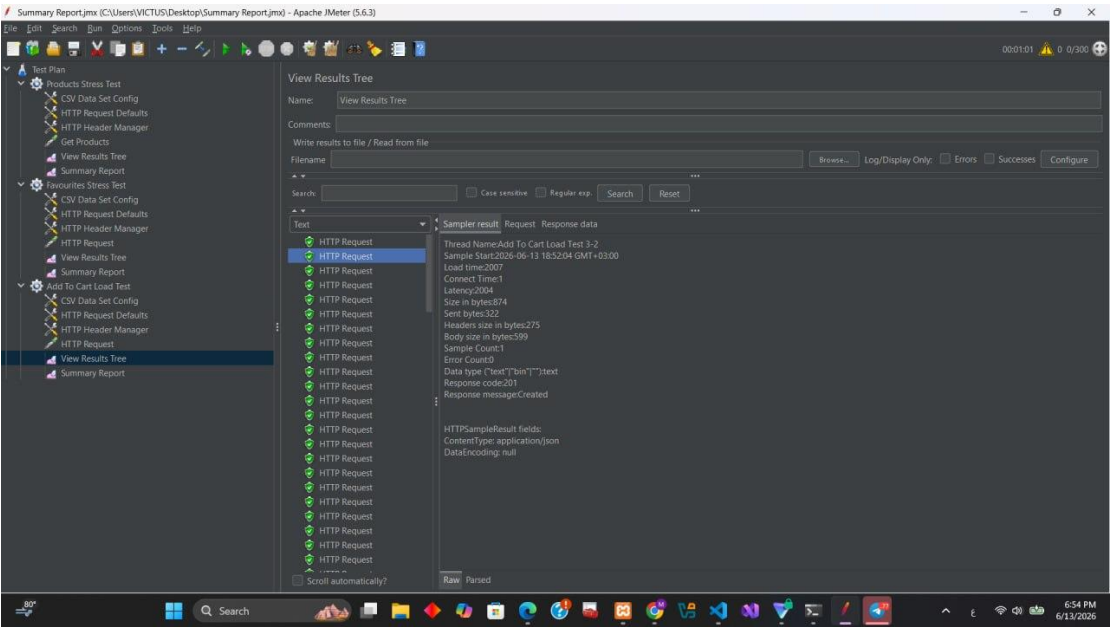
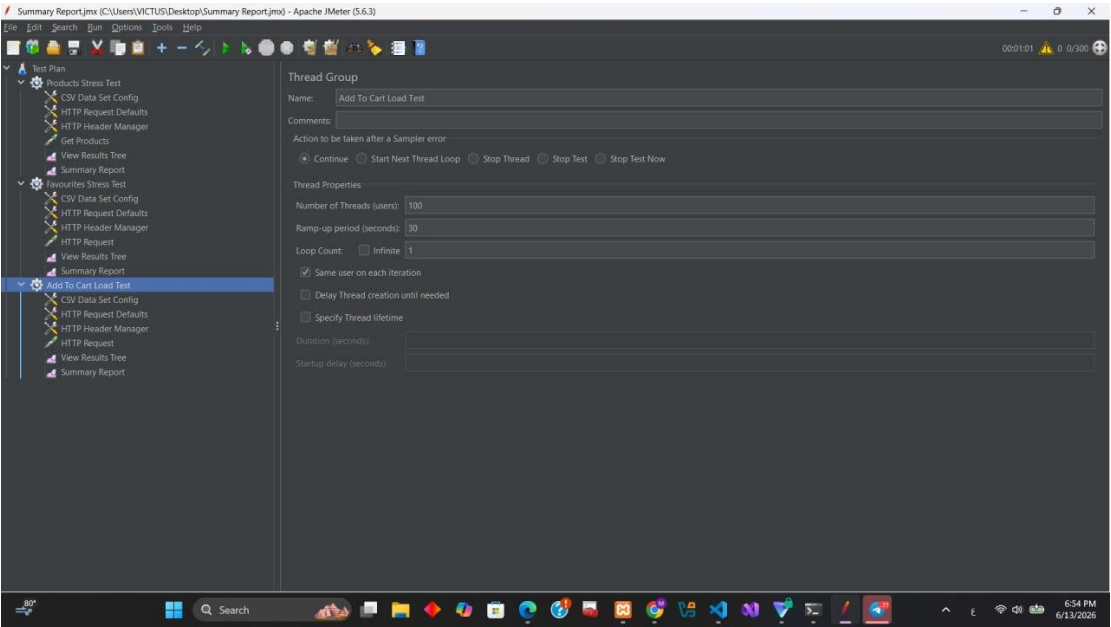
Duration (seconds):

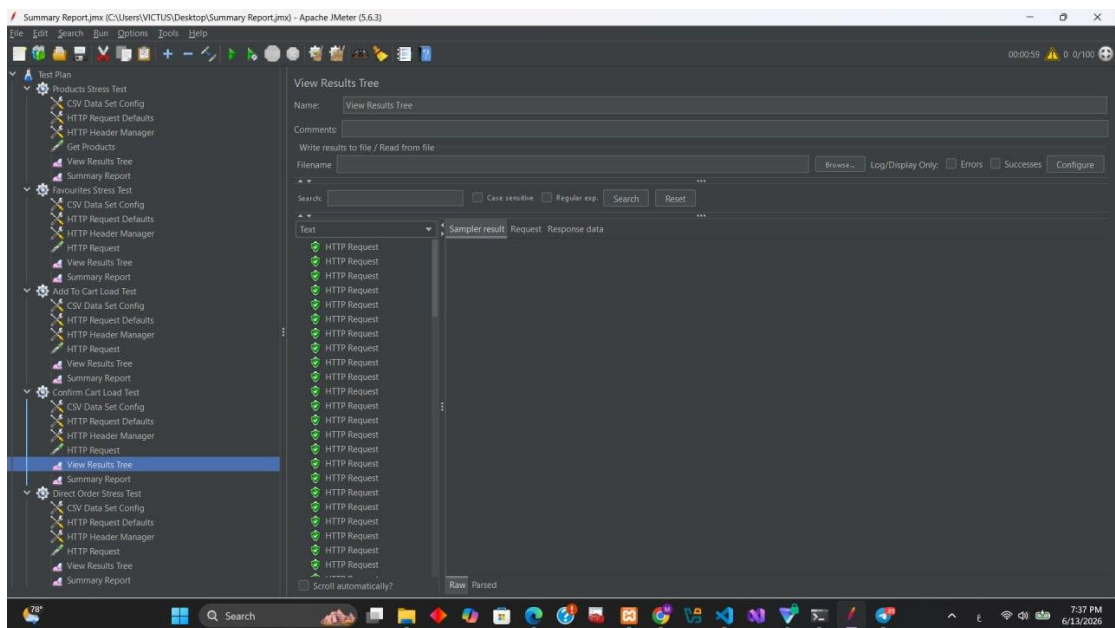
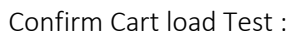
Startup delay (seconds):



Summary Report

Add to Cart Load Test :





Summary Report.jmx (C:\Users\WICTUS\Desktop\Summary Report.jmx) - Apache JMeter (5.6.3)

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	430	367	609	29.13	0.00%	1.7/sec	1.82	0.52	1116.8
TOTAL	100	430	367	609	29.13	0.00%	1.7/sec	1.82	0.52	1116.8

☐ Include group name in label? ☒ Save Table Header

Direct Order Stress Test :

Summary Report.jmx (C:\Users\WICTUS\Desktop\Summary Report.jmx) - Apache JMeter (5.6.3)

Thread Group

Name: Direct Order Stress Test

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users): 100

Ramp-up period (seconds): 60

Loop Count: ☐ Infinite ☐ 1

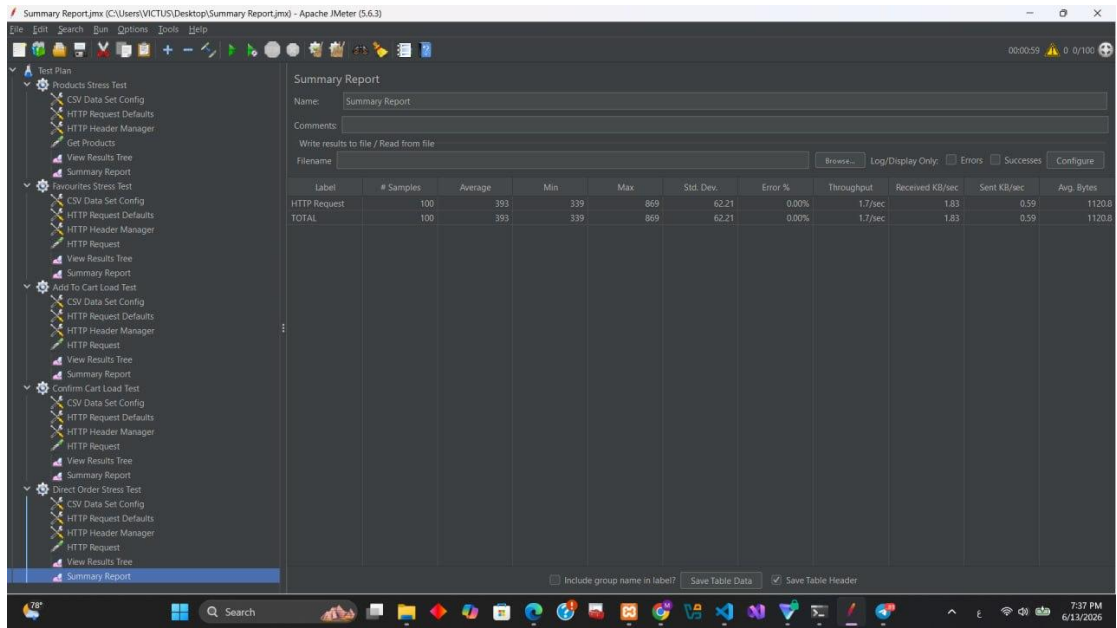
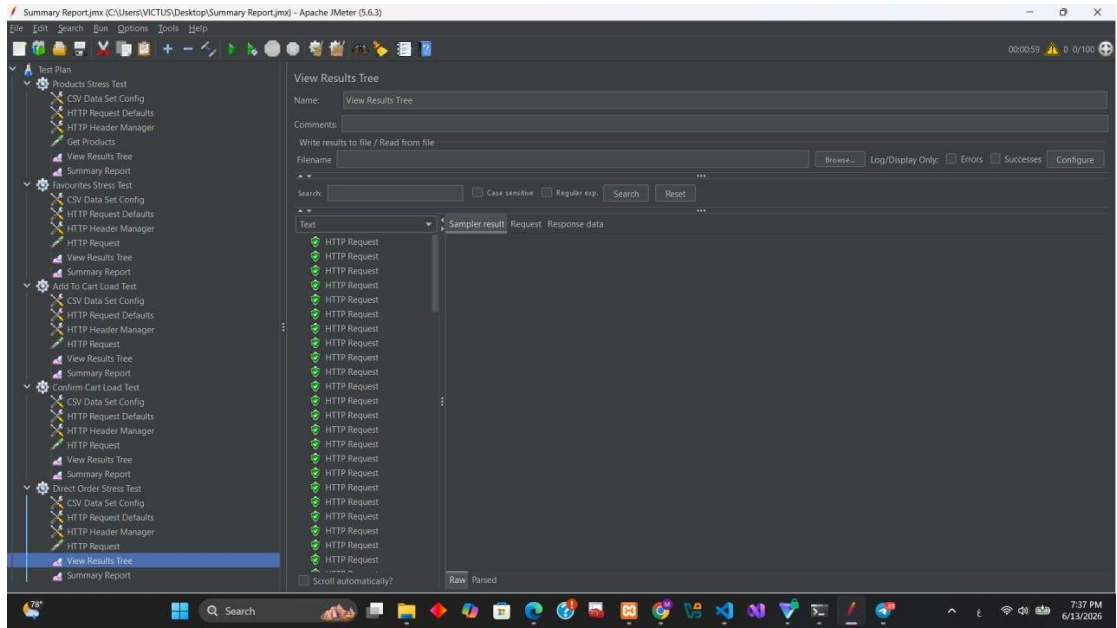
☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread Lifetime

Duration (seconds):

Startup delay (seconds):



Summary Report

Benchmarking & Bottleneck Analysis

وصف المشكلة

لا يكفي أن يعمل النظام بصورة صحيحة، بل يجب قياس أدائه وتحديد مواضع الاختناق التي تؤثر على سرعة الاستجابة تحت الحمل المرتفع.

الاستراتيجية المستخدمة

تم اختيار عملية عرض المنتجات باعتبارها من أكثر العمليات استخداماً داخل النظام. تم قياس أدائها قبل وبعد تطبيق Redis Cache بهدف تحديد الاختناق الأساسي وقياس أثر التحسين بصورة رقمية.

تحديد الاختناق

تبين أن الاختناق الرئيسي يتمثل في تكرار الاستعلامات المباشرة على قاعدة البيانات عند طلب البيانات نفسها بشكل متكرر من قبل عدد كبير من المستخدمين.

منهجية القياس

تم تنفيذ الاختبار مرتين:

• قبل تطبيق Redis Cache.

• بعد تطبيق Redis Cache.

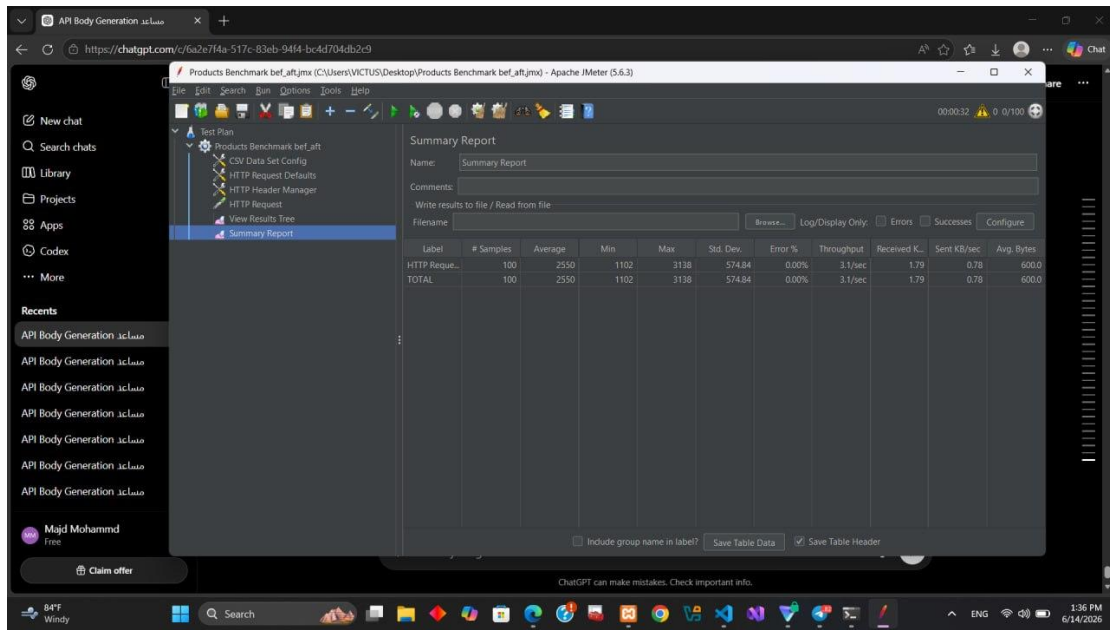
مع تثبيت جميع عوامل الاختبار الأخرى لضمان دقة المقارنة.

النتائج الرقمية

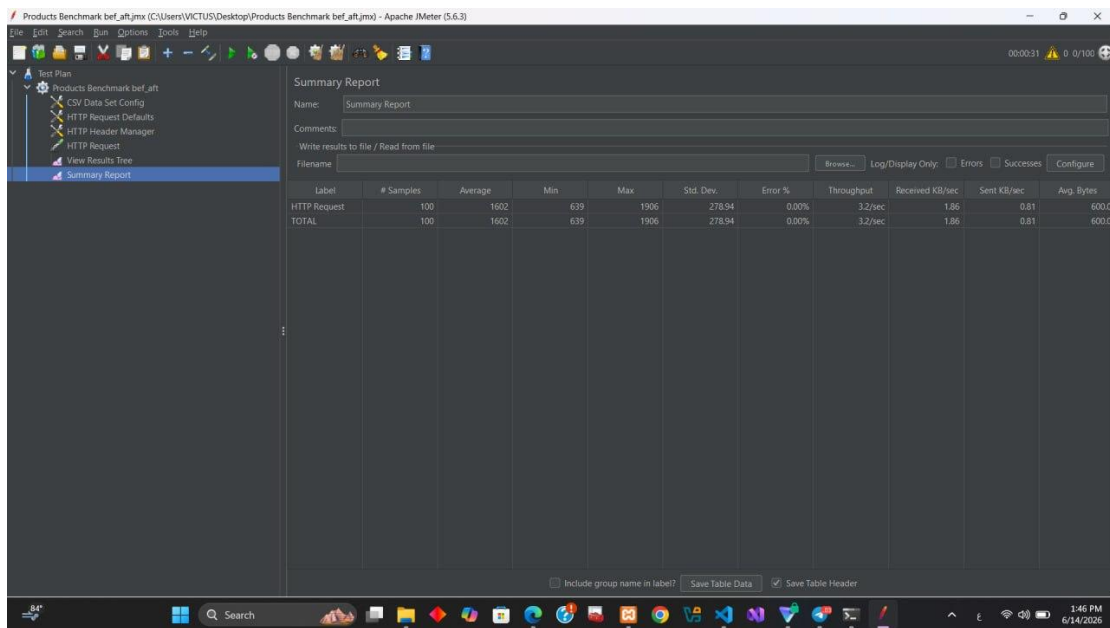
Metric	Before Cache	After Cache
Average Response Time	2550 ms	1602 ms
Minimum Response Time	1102 ms	639 ms
Maximum Response Time	3138 ms	1906 ms
Standard Deviation	574.84	278.94
Error Rate	0%	0%
Throughput	3.1/sec	3.2/sec

النتائج

أظهرت النتائج انخفاضاً ملحوظاً في زمن الاستجابة وتحسناً في استقرار الأداء بعد تطبيق Redis Cache ، مما يؤكد أن قاعدة البيانات كانت تمثل عنق الزجاجة الرئيسي في هذه العملية وأن استخدام التخزين المؤقت أدى إلى تخفيف هذا الاختناق بشكل واضح.



Before cache



After cache

تم تحديد الاختناق الرئيسي بنجاح، كما تم إثبات أثر التحسين بصورة رقمية من خلال مقارنة الأداء قبل وبعد تطبيق Redis Cache ، مما يحقق متطلبات Benchmarking and Bottleneck Analysis بصورة كاملة.

الخاتمة

يهدف هذا المشروع إلى تطبيق مفاهيم البرمجة المتوازية والأنظمة الموزعة ضمن نظام تجارة إلكترونية متكامل باستخدام إطار العمل Laravel. خلال مراحل التطوير تم التركيز على تحقيق مجموعة من المتطلبات الوظيفية وغير الوظيفية المتعلقة بالتزامن، سلامة البيانات، الأداء، وقابلية التوسع.

تم تصميم وتنفيذ آليات متعددة لمعالجة المشكلات الشائعة في الأنظمة المتزامنة، بما في ذلك حماية الموارد المشتركة من التضارب، استخدام الأقفال الموزعة للتحكم بالوصول إلى المخزون الحساس، وضمان سلامة المعاملات باستخدام المعاملات الذرية (Transactions) بحيث تنجح العمليات المركبة بالكامل أو تفشل بالكامل دون ترك بيانات غير متسقة.

كما تم دمج طبقة تخزين مؤقت باستخدام Redis لتقليل الضغط على قاعدة البيانات وتحسين أداء العمليات الأكثر استخدامًا داخل النظام. بالإضافة إلى ذلك تم تنفيذ محاكاة لتوزيع الطلبات على عدة خوادم باستخدام استراتيجية Round Robin بهدف توضيح أحد الأساليب الشائعة في موازنة الأحمال.

وللتحقق من كفاءة الحلول المطبقة، تم إجراء مجموعة واسعة من اختبارات الأداء باستخدام Apache JMeter، شملت اختبارات التزامن، اختبارات الضغط لمحاكاة 100 مستخدم متزامن، بالإضافة إلى اختبارات القياس وتحديد الاختناقات (Benchmarking & Bottleneck Analysis). وقد أظهرت النتائج قدرة النظام على التعامل مع الأحمال المتزامنة والمحافظة على سلامة البيانات مع تحقيق تحسن ملحوظ في زمن الاستجابة بعد تطبيق تقنيات التحسين المناسبة.

في النهاية، نجح المشروع في تحقيق جميع المتطلبات المطلوبة، كما قدم نموذجًا عمليًا يوضح كيفية توظيف مفاهيم البرمجة المتوازية والأنظمة الموزعة داخل تطبيق حقيقي مع المحافظة على الأداء والموثوقية وسلامة البيانات.